Lab 3: Human-in-the-Loop Product Review Moderation - Full Implementation Guide

1. Overview

This document explains the complete Lab 3 implementation and includes all scripts. The system moderates product reviews with a Human-in-the-Loop (HITL) checkpoint:

1. AI analyses review text against retrieved moderation policies.
2. System pauses for a human moderator decision using LangGraph interrupt.
3. Human can APPROVE, OVERRIDE, or ESCALATE.
4. Final decision is logged with a full audit trail.
5. Escalations are additionally logged to a separate escalation file.

2. Architecture and Flow

The graph pipeline is:

START -> AnalyseReviewNode -> PresentForModerationNode (interrupt) -> HumanModerationNode

Routing after human moderation:

- If input is invalid: route back to PresentForModerationNode and re-prompt.
- If ESCALATE: route to EscalationNode (writes escalations.json), then to RecordDecisionNode.
- Otherwise: route directly to RecordDecisionNode.

This guarantees no final moderation decision is written until human action is received.

3. Step-by-Step Implementation Notes

Step 1: Define state schema and graph orchestration

In graph.py we define a TypedDict state with fields required by the lab: - review_text, review_id - retrieved_policies - ai_recommendation, ai_reasoning, violated_rules, confidence - human_decision, final_action, moderator_note - escalation flags and logging status fields

MemorySaver is used so the graph state persists across interrupt/resume.

Step 2: Build AnalyseReviewNode

In nodes/analyse.py: - Open ChromaDB collection moderation_policies persisted at ./chroma_db. - Retrieve top policy snippets using semantic search against review text. - Build a strict JSON prompt for Ollama to keep output machine-parseable. - Parse and normalize output into action, confidence, violated rules,

and reasoning. - If runtime errors occur, fail safely with action FLAG and fallback reasoning.

Step 3: Build PresentForModerationNode with interrupt()

In nodes/present.py: - Render a complete moderation panel including review text, AI output, confidence, and policy context. - Call interrupt(panel_text) to pause graph execution. - Resume path expects graph.invoke(Command(resume=human_input), config=thread_config).

Step 4: Build HumanModerationNode input validation

In nodes/moderate.py: - Parse moderator commands strictly. - APPROVE: accept AI recommendation. - OVERRIDE requires both a valid action and a reason. - ESCALATE requires a non-empty senior note. - Invalid input sets validation_error to force a re-prompt loop.

Step 5: Build audit logging nodes

In nodes/record.py: - record_escalation writes escalations.json for ESCALATE decisions. - record_decision writes moderation_log.json for every finalized decision. - Both logs include timestamped context for accountability.

Step 6: Build ingestion script and policy corpus

In ingest.py: - Load policy files from data/ecommerce_policies. - Derive metadata {policy_id, category, severity} from filename pattern. - Upsert into ChromaDB collection moderation_policies. - Use –reset to recreate collection for clean reruns.

Step 7: Provide example logs and execution guide

- moderation_log.json includes sample APPROVE, OVERRIDE, and ESCALATE outcomes.
- escalations.json includes a sample escalated record.
- README.md explains setup and run commands.

4. How to Run

```
conda activate agentic
cd /home/chaitanya-kohli/AgenticLab_Kohli/lab3_hitl
python ingest.py --data-dir data/ecommerce_policies --chroma-dir chroma_db --reset
python graph.py REV-2001 "This seller posted fake specs and spam links"
```

At pause prompt, enter one of:

- APPROVE
- OVERRIDE FLAG Reason text here

- ESCALATE Requires senior fraud analyst review

5. Full Script Listings

File: graph.py

```
"""Lab 3 HITL moderation graph.

Graph flow:
START -> analyse -> present (interrupt) -> moderate
  -> if invalid input: present (re-prompt)
  -> if ESCALATE: escalate -> record
  -> else: record
record -> END

This design guarantees that no moderation decision is written before human action.
"""

from __future__ import annotations

from typing import Any
from typing import TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command

from nodes import (
    analyse_review,
    human_moderation,
    present_for_moderation,
    record_decision,
    record_escalation,
)


class ReviewModerationState(TypedDict, total=False):
    """State schema required by the lab handbook and routing logic."""

    review_text: str
    review_id: str
    retrieved_policies: list[dict[str, Any]]
    ai_recommendation: str
    ai_reasoning: str
```

```
    violated_rules: list[str]
    confidence: float

    human_decision: str
    final_action: str
    moderator_note: str

    escalation_required: bool
    escalation_logged: bool
    decision_logged: bool
    validation_error: str


def _route_after_moderation(state: ReviewModerationState) -> str:
    """Conditional router after human input parsing."""
    if state.get("validation_error"):
        # Invalid moderator input loops back to PresentForModerationNode.
        return "present"

    if state.get("escalation_required", False):
        return "escalate"

    return "record"


def build_graph():
    """Compile and return the HITL graph with MemorySaver checkpointing."""
    graph_builder = StateGraph(ReviewModerationState)

    graph_builder.add_node("analyse", analyse_review)
    graph_builder.add_node("present", present_for_moderation)
    graph_builder.add_node("moderate", human_moderation)
    graph_builder.add_node("escalate", record_escalation)
    graph_builder.add_node("record", record_decision)

    graph_builder.add_edge(START, "analyse")
    graph_builder.add_edge("analyse", "present")
    graph_builder.add_edge("present", "moderate")

    graph_builder.add_conditional_edges(
        "moderate",
        _route_after_moderation,
        {
            "present": "present",
            "escalate": "escalate",
            "record": "record",
```

```
        },
    )

    graph_builder.add_edge("escalate", "record")
    graph_builder.add_edge("record", END)

    # MemorySaver ensures state survives interrupt() pause/resume cycles.
    return graph_builder.compile(checkpointer=MemorySaver())


def run_hitl_session(review_id: str, review_text: str, thread_id: str = "lab3-session") -> d
    """Run a complete interactive HITL moderation session from terminal.

    The function invokes the graph once, then keeps resuming from interrupts until
    a final state is produced.
    """
    graph = build_graph()
    config = {"configurable": {"thread_id": thread_id}}

    state: ReviewModerationState = {
        "review_id": review_id,
        "review_text": review_text,
    }

    result = graph.invoke(state, config=config)

    while "__interrupt__" in result:
        interrupt_payload = result["__interrupt__"][0].value
        print("\n" + str(interrupt_payload) + "\n")

        moderator_input = input("Moderator decision: ").strip()
        result = graph.invoke(Command(resume=moderator_input), config=config)

    return result


if __name__ == "__main__":
    import argparse
    import json

    parser = argparse.ArgumentParser(description="Run Lab 3 HITL moderation flow.")
    parser.add_argument("review_id", type=str, help="Unique identifier for the review")
    parser.add_argument("review_text", type=str, help="The review text to moderate")
    parser.add_argument(
        "--thread-id",
        type=str,
```

```
        default="lab3-session",
        help="Thread ID for LangGraph checkpoint state",
    )

    args = parser.parse_args()
    final_state = run_hitl_session(
        review_id=args.review_id,
        review_text=args.review_text,
        thread_id=args.thread_id,
    )

    print("Final state:")
    print(json.dumps(final_state, indent=2, default=str))
```

File: ingest.py

```
"""Policy ingestion script for Lab 3.

This script loads policy documents into the ChromaDB collection named
"moderation_policies" and persists the vector DB at ./chroma_db.
"""

from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path
from typing import Any

import chromadb
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunction

COLLECTION_NAME = "moderation_policies"
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"


def _parse_metadata_from_filename(file_path: Path) -> dict[str, Any]:
    """Infer metadata from file name pattern: <category>__<severity>__<id>.txt.

    If the pattern is not present, safe defaults are used.
    """
    stem = file_path.stem
    parts = stem.split("__")
```

```
    category = parts[0] if len(parts) > 0 and parts[0] else "general"
    severity = parts[1] if len(parts) > 1 and parts[1] else "medium"
    policy_id = parts[2] if len(parts) > 2 and parts[2] else stem

    return {
        "policy_id": policy_id,
        "category": category,
        "severity": severity,
    }


def _read_policy_document(file_path: Path) -> tuple[str, dict[str, Any]]:
    """Read one policy file and return (content, metadata)."""
    text = file_path.read_text(encoding="utf-8").strip()
    metadata = _parse_metadata_from_filename(file_path)

    # Optional frontmatter support: a leading JSON line can override metadata.
    # Example first line: {"policy_id":"P123","category":"spam","severity":"high"}
    lines = text.splitlines()
    if lines:
        first_line = lines[0].strip()
        if first_line.startswith("{") and first_line.endswith("}"):
            try:
                override = json.loads(first_line)
                if isinstance(override, dict):
                    metadata.update({
                        "policy_id": override.get("policy_id", metadata["policy_id"]),
                        "category": override.get("category", metadata["category"]),
                        "severity": override.get("severity", metadata["severity"]),
                    })
                    text = "\n".join(lines[1:]).strip()
            except json.JSONDecodeError:
                # Ignore malformed frontmatter and keep full original text.
                pass

    return text, metadata


def _build_collection(chroma_dir: Path):
    """Create/open the moderation_policies collection with embedding function."""
    embedding_fn = SentenceTransformerEmbeddingFunction(model_name=EMBEDDING_MODEL)
    client = chromadb.PersistentClient(path=str(chroma_dir))
    return client.get_or_create_collection(
        name=COLLECTION_NAME,
        embedding_function=embedding_fn,
```

```
            metadata={"created_by": "lab3_ingest"},
        )


    def ingest_policies(data_dir: Path, chroma_dir: Path, reset: bool = False) -> None:
        """Ingest all policy docs from data_dir into ChromaDB."""
        if not data_dir.exists():
            raise FileNotFoundError(f"Policy directory not found: {data_dir}")

        files = sorted(
            [p for p in data_dir.rglob("*") if p.suffix.lower() in {".txt", ".md"}]
        )
        if len(files) < 15:
            print(
                "Warning: fewer than 15 policy files found. "
                "Lab requirement recommends 15+ documents."
            )

        collection = _build_collection(chroma_dir)

        if reset:
            # Reset by deleting and recreating the collection.
            client = chromadb.PersistentClient(path=str(chroma_dir))
            try:
                client.delete_collection(COLLECTION_NAME)
            except Exception:
                pass
            collection = _build_collection(chroma_dir)

        ids: list[str] = []
        documents: list[str] = []
        metadatas: list[dict[str, Any]] = []

        for file_path in files:
            text, metadata = _read_policy_document(file_path)
            if not text:
                continue

            # Stable IDs make repeated ingest idempotent.
            digest = hashlib.md5(str(file_path).encode("utf-8")).hexdigest()[:12]
            doc_id = f"{metadata['policy_id']}_{digest}"

            ids.append(doc_id)
            documents.append(text)
            metadatas.append(metadata)
```

```
    if not documents:
        raise ValueError("No valid policy documents found to ingest.")

    collection.upsert(ids=ids, documents=documents, metadatas=metadatas)
    print(f"Ingested {len(documents)} policy documents into '{COLLECTION_NAME}'.")
    print(f"ChromaDB persist directory: {chroma_dir}")


def main() -> None:
    """CLI entry point for policy ingestion."""
    parser = argparse.ArgumentParser(description="Ingest Lab 3 moderation policies.")
    parser.add_argument(
        "--data-dir",
        type=Path,
        default=Path("data/ecommerce_policies"),
        help="Directory containing policy .txt/.md files",
    )
    parser.add_argument(
        "--chroma-dir",
        type=Path,
        default=Path("chroma_db"),
        help="Persist directory for ChromaDB",
    )
    parser.add_argument(
        "--reset",
        action="store_true",
        help="Delete and recreate moderation_policies before ingest",
    )

    args = parser.parse_args()
    ingest_policies(args.data_dir, args.chroma_dir, reset=args.reset)


if __name__ == "__main__":
    main()
```

File: nodes/analyse.py

```
"""AnalyseReviewNode for Lab 3 HITL moderation pipeline.

This module handles three responsibilities:
1) Retrieve relevant moderation policy snippets from ChromaDB.
2) Ask the local Ollama model to analyse a review against those snippets.
3) Return structured fields that downstream HITL nodes can display and moderate.
```

```
"""

from __future__ import annotations

import json
import os
import re
from typing import Any

import chromadb
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunction
from dotenv import load_dotenv
from langchain_community.llms import Ollama

# Load environment values like OLLAMA_BASE_URL and OLLAMA_MODEL from .env.
load_dotenv()

COLLECTION_NAME = "moderation_policies"
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
DEFAULT_PERSIST_DIR = os.getenv("CHROMA_PERSIST_DIR", "chroma_db")
OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3")


def _build_policy_collection(persist_dir: str = DEFAULT_PERSIST_DIR):
    """Return the moderation_policies ChromaDB collection.

    The collection is read-only during analysis; this function only opens it.
    """
    embedding_fn = SentenceTransformerEmbeddingFunction(model_name=EMBEDDING_MODEL)
    client = chromadb.PersistentClient(path=persist_dir)
    return client.get_or_create_collection(
        name=COLLECTION_NAME,
        embedding_function=embedding_fn,
        metadata={"purpose": "Lab3 moderation policy retrieval"},
    )


def _format_retrieved_policies(chroma_result: dict[str, Any]) -> list[dict[str, Any]]:
    """Convert raw ChromaDB query output into a clean list of policy dictionaries."""
    documents = chroma_result.get("documents", [[]])[0]
    metadatas = chroma_result.get("metadatas", [[]])[0]
    distances = chroma_result.get("distances", [[]])[0]
    ids = chroma_result.get("ids", [[]])[0]

    formatted: list[dict[str, Any]] = []
```

```
    for idx, text in enumerate(documents):
        # Distance can be missing depending on ChromaDB version/config.
        distance = distances[idx] if idx < len(distances) else None
        metadata = metadatas[idx] if idx < len(metadatas) and metadatas[idx] else {}
        doc_id = ids[idx] if idx < len(ids) else f"policy_{idx}"

        formatted.append(
            {
                "id": doc_id,
                "text": text,
                "category": metadata.get("category", "general"),
                "severity": metadata.get("severity", "medium"),
                "policy_id": metadata.get("policy_id", doc_id),
                "distance": distance,
            }
        )

    return formatted


def _build_analysis_prompt(review_text: str, policies: list[dict[str, Any]]) -> str:
    """Create a strict JSON-output prompt for the local moderation model."""
    policy_lines = []
    for policy in policies:
        policy_lines.append(
            f"- PolicyID={policy['policy_id']} | Category={policy['category']}"
            f" | Severity={policy['severity']} | Text={policy['text']}"
        )

    joined_policies = "\n".join(policy_lines) if policy_lines else "- No policies retrieved.

    # The model is instructed to return JSON only so parsing remains reliable.
    return f"""
You are a moderation analyst for an e-commerce platform.
Given the user review and retrieved policy snippets, produce one JSON object only.

Allowed final actions: APPROVE, FLAG, REMOVE

Review:
{review_text}

Retrieved policies:
{joined_policies}

Return STRICT JSON only with this exact schema:
{{
```

```
  "action": "APPROVE|FLAG|REMOVE",
  "confidence": 0.0,
  "violated_rules": ["policy_id_or_rule_name"],
  "reasoning": "short but specific explanation"
}}

Rules:
- confidence must be a float between 0 and 1.
- If uncertain, prefer action=FLAG.
- violated_rules should include policy IDs from retrieved policies when applicable.
- Do not output markdown, prose, or code fences.
""".strip()


def _extract_first_json_object(text: str) -> str:
    """Extract the first JSON object from model text.

    Some local models occasionally add preface/postface text despite instructions.
    This helper salvages the first top-level JSON object from the response.
    """
    text = text.strip()
    if text.startswith("{") and text.endswith("}"):
        return text

    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        raise ValueError("No JSON object found in model output.")
    return match.group(0)


def _parse_model_response(response_text: str) -> dict[str, Any]:
    """Parse model response into canonical moderation fields with safe defaults."""
    raw_json = _extract_first_json_object(response_text)
    parsed = json.loads(raw_json)

    action = str(parsed.get("action", "FLAG")).upper().strip()
    if action not in {"APPROVE", "FLAG", "REMOVE"}:
        action = "FLAG"

    confidence = parsed.get("confidence", 0.5)
    try:
        confidence = float(confidence)
    except (TypeError, ValueError):
        confidence = 0.5
    confidence = max(0.0, min(1.0, confidence))
```

```
    violated_rules = parsed.get("violated_rules", [])
    if not isinstance(violated_rules, list):
        violated_rules = []

    reasoning = str(parsed.get("reasoning", "No reasoning provided.")).strip()

    return {
        "action": action,
        "confidence": confidence,
        "violated_rules": violated_rules,
        "reasoning": reasoning,
    }


def analyse_review(state: dict[str, Any]) -> dict[str, Any]:
    """AnalyseReviewNode implementation.

    Input state (required):
    - review_text: str

    Returns state updates:
    - retrieved_policies: list[dict]
    - ai_recommendation: APPROVE|FLAG|REMOVE
    - ai_reasoning: str
    - violated_rules: list[str]
    - confidence: float
    """
    review_text = str(state.get("review_text", "")).strip()
    if not review_text:
        # Fail safely by flagging empty submissions.
        return {
            "retrieved_policies": [],
            "ai_recommendation": "FLAG",
            "ai_reasoning": "Review text is empty; cannot analyse content safely.",
            "violated_rules": ["missing_review_text"],
            "confidence": 1.0,
        }

    try:
        collection = _build_policy_collection()
        query_result = collection.query(query_texts=[review_text], n_results=5)
        retrieved_policies = _format_retrieved_policies(query_result)

        llm = Ollama(
            base_url=OLLAMA_BASE_URL,
            model=OLLAMA_MODEL,
```

```
            temperature=0.0,
        )
        prompt = _build_analysis_prompt(review_text, retrieved_policies)
        llm_response = llm.invoke(prompt)

        parsed = _parse_model_response(str(llm_response))
        return {
            "retrieved_policies": retrieved_policies,
            "ai_recommendation": parsed["action"],
            "ai_reasoning": parsed["reasoning"],
            "violated_rules": parsed["violated_rules"],
            "confidence": parsed["confidence"],
        }

    except Exception as exc:
        # Never crash the pipeline in production moderation flows.
        return {
            "retrieved_policies": [],
            "ai_recommendation": "FLAG",
            "ai_reasoning": f"Analysis fallback due to runtime error: {exc}",
            "violated_rules": ["analysis_runtime_error"],
            "confidence": 0.2,
        }
```

File: nodes/present.py

```
"""PresentForModerationNode for Lab 3 HITL moderation pipeline.

This node is the explicit human-in-the-loop checkpoint.
It renders the AI recommendation, then pauses graph execution using interrupt().
"""

from __future__ import annotations

from typing import Any

from langgraph.types import interrupt


def _format_policy_summary(retrieved_policies: list[dict[str, Any]]) -> str:
    """Build a compact policy summary for the moderator console view."""
    if not retrieved_policies:
        return "No policies retrieved for this review."
```

```
    lines: list[str] = []
    for policy in retrieved_policies[:5]:
        lines.append(
            f"- {policy.get('policy_id', 'unknown')}"
            f" ({policy.get('category', 'general')}, {policy.get('severity', 'medium')})"
        )
    return "\n".join(lines)


def _render_panel(state: dict[str, Any]) -> str:
    """Render a human-friendly moderation panel that includes input instructions."""
    review_id = state.get("review_id", "unknown_review")
    review_text = state.get("review_text", "")
    ai_recommendation = state.get("ai_recommendation", "FLAG")
    confidence = state.get("confidence", 0.0)
    violated_rules = state.get("violated_rules", [])
    ai_reasoning = state.get("ai_reasoning", "")
    validation_error = state.get("validation_error", "")

    policy_summary = _format_policy_summary(state.get("retrieved_policies", []))
    violated = ", ".join(violated_rules) if violated_rules else "None"

    error_block = ""
    if validation_error:
        # When moderator input is invalid, we show exactly what was wrong and re-prompt.
        error_block = f"\nINPUT ERROR: {validation_error}\n"

    return f"""
================ HUMAN MODERATION CHECKPOINT ================
Review ID: {review_id}

Review Text:
{review_text}

AI Recommendation: {ai_recommendation}
AI Confidence: {confidence}
Violated Rules: {violated}

AI Reasoning:
{ai_reasoning}

Retrieved Policy Context:
{policy_summary}
{error_block}
Enter one of the following commands:
1) APPROVE
```

```
2) OVERRIDE <APPROVE|FLAG|REMOVE> <reason>
3) ESCALATE <senior_moderator_note>
================================================================
""".strip()


def present_for_moderation(state: dict[str, Any]) -> dict[str, Any]:
    """Pause the graph and collect a human moderator decision via interrupt()."""
    panel_text = _render_panel(state)

    # interrupt() freezes execution and returns control to the caller.
    # The graph resumes when the caller invokes Command(resume=<human_input>).
    human_input = interrupt(panel_text)

    return {
        "human_decision": str(human_input).strip(),
        # Clear previous validation error after a new input arrives.
        "validation_error": "",
    }
```

File: nodes/moderate.py

```
"""HumanModerationNode for Lab 3 HITL moderation pipeline.

This node validates moderator input and normalizes it into fields used by routing
and logging nodes. Invalid input triggers a re-prompt in the graph.
"""

from __future__ import annotations

from typing import Any

ALLOWED_ACTIONS = {"APPROVE", "FLAG", "REMOVE"}


def _parse_human_decision(raw_input: str, ai_recommendation: str) -> dict[str, Any]:
    """Parse and validate moderator commands.

    Supported commands:
    - APPROVE
    - OVERRIDE <APPROVE|FLAG|REMOVE> <reason>
    - ESCALATE <senior_moderator_note>
    """
    text = (raw_input or "").strip()
```

```
if not text:
    return {
        "validation_error": "Empty input. Please enter APPROVE, OVERRIDE, or ESCALATE.",
    }

tokens = text.split()
command = tokens[0].upper()

if command == "APPROVE":
    # APPROVE means accept the AI recommendation as final.
    final_action = ai_recommendation if ai_recommendation in ALLOWED_ACTIONS else "FLAG"
    return {
        "parsed_human_action": "APPROVE",
        "final_action": final_action,
        "moderator_note": "Moderator approved AI recommendation.",
        "escalation_required": False,
        "validation_error": "",
    }

if command == "OVERRIDE":
    if len(tokens) < 3:
        return {
            "validation_error": (
                "OVERRIDE requires both a new action and a reason. "
                "Format: OVERRIDE <APPROVE|FLAG|REMOVE> <reason>"
            )
        }

    requested_action = tokens[1].upper().strip()
    reason = " ".join(tokens[2:]).strip()

    if requested_action not in ALLOWED_ACTIONS:
        return {
            "validation_error": (
                "Invalid OVERRIDE action. Use APPROVE, FLAG, or REMOVE."
            )
        }

    if not reason:
        return {
            "validation_error": "OVERRIDE reason cannot be empty.",
        }

    return {
        "parsed_human_action": "OVERRIDE",
        "final_action": requested_action,
```

```
            "moderator_note": f"Moderator override reason: {reason}",
            "escalation_required": False,
            "validation_error": "",
        }

    if command == "ESCALATE":
        note = " ".join(tokens[1:]).strip()
        if not note:
            return {
                "validation_error": "ESCALATE requires a senior moderator note.",
            }

        return {
            "parsed_human_action": "ESCALATE",
            "final_action": "ESCALATE",
            "moderator_note": note,
            "escalation_required": True,
            "validation_error": "",
        }

    return {
        "validation_error": (
            "Unknown command. Use APPROVE, OVERRIDE <action> <reason>, "
            "or ESCALATE <note>."
        )
    }


def human_moderation(state: dict[str, Any]) -> dict[str, Any]:
    """HumanModerationNode entry point.

    It parses moderator input and returns graph state updates.
    """
    human_decision = str(state.get("human_decision", ""))
    ai_recommendation = str(state.get("ai_recommendation", "FLAG")).upper()
    return _parse_human_decision(human_decision, ai_recommendation)
```

File: nodes/record.py

```
"""RecordDecisionNode and EscalationNode for Lab 3 HITL moderation pipeline.

These nodes are the audit layer for moderation governance.
- record_escalation writes escalated reviews to escalations.json
- record_decision writes all finalized reviews to moderation_log.json
```

```
"""

from __future__ import annotations

import json
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

MODERATION_LOG_PATH = Path("moderation_log.json")
ESCALATION_LOG_PATH = Path("escalations.json")


def _now_iso() -> str:
    """Return UTC timestamp in ISO-8601 format for audit consistency."""
    return datetime.now(timezone.utc).isoformat()


def _read_json_list(file_path: Path) -> list[dict[str, Any]]:
    """Read a JSON array from disk; if absent or invalid, return an empty list."""
    if not file_path.exists():
        return []

    try:
        data = json.loads(file_path.read_text(encoding="utf-8"))
        if isinstance(data, list):
            return data
        return []
    except json.JSONDecodeError:
        return []


def _write_json_list(file_path: Path, data: list[dict[str, Any]]) -> None:
    """Write a JSON array with indentation for human-readable audit trails."""
    file_path.write_text(json.dumps(data, indent=2), encoding="utf-8")


def _build_base_audit_entry(state: dict[str, Any]) -> dict[str, Any]:
    """Build common fields used by both escalation and moderation logs."""
    return {
        "timestamp": _now_iso(),
        "review_id": state.get("review_id", "unknown_review"),
        "review_text": state.get("review_text", ""),
        "ai_recommendation": state.get("ai_recommendation", "FLAG"),
        "ai_reasoning": state.get("ai_reasoning", ""),
        "violated_rules": state.get("violated_rules", []),
```

```
        "confidence": state.get("confidence", 0.0),
        "human_decision": state.get("human_decision", ""),
        "final_action": state.get("final_action", "FLAG"),
        "moderator_note": state.get("moderator_note", ""),
    }


def record_escalation(state: dict[str, Any]) -> dict[str, Any]:
    """Write an escalation-specific entry when moderator selects ESCALATE."""
    entry = _build_base_audit_entry(state)
    entry["escalated"] = True

    escalations = _read_json_list(ESCALATION_LOG_PATH)
    escalations.append(entry)
    _write_json_list(ESCALATION_LOG_PATH, escalations)

    return {"escalation_logged": True}


def record_decision(state: dict[str, Any]) -> dict[str, Any]:
    """Write the final moderation decision to moderation_log.json.

    This node should run only after validated human input is available.
    """
    entry = _build_base_audit_entry(state)
    entry["escalated"] = bool(state.get("escalation_required", False))

    moderation_log = _read_json_list(MODERATION_LOG_PATH)
    moderation_log.append(entry)
    _write_json_list(MODERATION_LOG_PATH, moderation_log)

    return {"decision_logged": True}
```

File: nodes/init.py

```
"""Node package exports for Lab 3 HITL pipeline."""

from .analyse import analyse_review
from .moderate import human_moderation
from .present import present_for_moderation
from .record import record_decision, record_escalation

__all__ = [
    "analyse_review",
```

```
    "present_for_moderation",
    "human_moderation",
    "record_escalation",
    "record_decision",
]
```

File: README.md

```
# Lab 3: Human-in-the-Loop Product Review Moderation

This project implements a LangGraph HITL moderation pipeline for e-commerce reviews.
It follows the lab requirements:

- Uses `StateGraph` with a typed state schema.
- Uses `interrupt()` in `PresentForModerationNode`.
- Uses `MemorySaver` to persist state through pause/resume.
- Enforces moderator commands:
  - `APPROVE`
  - `OVERRIDE <APPROVE|FLAG|REMOVE> <reason>`
  - `ESCALATE <senior_note>`
- Logs final decisions to `moderation_log.json`.
- Logs escalations to `escalations.json` via a dedicated escalation node.

## Project Structure

- `graph.py`: Full LangGraph orchestration with pause/resume and routing.
- `nodes/analyse.py`: Retrieval + AI policy analysis.
- `nodes/present.py`: Human checkpoint with `interrupt()`.
- `nodes/moderate.py`: Human command validation and normalization.
- `nodes/record.py`: Decision and escalation audit logging.
- `ingest.py`: Policy document ingestion to ChromaDB collection `moderation_policies`.
- `.env`: Ollama + Chroma configuration.
- `moderation_log.json`: Audit log of moderation decisions.
- `escalations.json`: Dedicated escalation log.
- `data/ecommerce_policies/`: Local sample policy documents.

## Run Steps

1. Activate your environment (already created by you):

```bash
conda activate agentic
```

2. Go to the project folder:
```

```
cd /home/chaitanya-kohli/AgenticLab_Kohli/lab3_hitl
```

3. Ingest policy documents (one-time setup):

```
python ingest.py --data-dir data/ecommerce_policies --chroma-dir chroma_db --reset
```

4. Run a moderation session:

```
python graph.py REV-2001 "This is the worst product, seller is a scammer"
```

5. When prompted at the checkpoint, enter one of:

- `APPROVE`
- `OVERRIDE FLAG The review is harsh but not policy violating`
- `ESCALATE Needs senior fraud analyst review`

How Pause/Resume Works

- `present_for_moderation()` calls `interrupt(panel_text)`.
- Graph execution pauses and returns control to caller.
- Caller resumes with:

```
graph.invoke(Command(resume=human_input), config=thread_config)
```

- State is preserved by `MemorySaver`, so analysis is not recomputed on resume.

Notes

- The collection name is fixed to `moderation_policies` as required.
- The Chroma persist path defaults to `./chroma_db`.
- No cloud API keys are required; local Ollama is used.

File: .env

```dotenv
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3
CHROMA_PERSIST_DIR=chroma_db
```

File: moderation_log.json


```
[
  {
    "timestamp": "2026-06-28T09:00:00+00:00",
    "review_id": "REV-1001",
```

```
    "review_text": "Great product, fast delivery, highly recommend!",
    "ai_recommendation": "APPROVE",
    "ai_reasoning": "No policy violations detected; positive experience review.",
    "violated_rules": [],
    "confidence": 0.93,
    "human_decision": "APPROVE",
    "final_action": "APPROVE",
    "moderator_note": "Moderator approved AI recommendation.",
    "escalated": false
  },
  {
    "timestamp": "2026-06-28T09:15:00+00:00",
    "review_id": "REV-1002",
    "review_text": "This seller is fake, and this product is a total scam.",
    "ai_recommendation": "REMOVE",
    "ai_reasoning": "Potential defamation and abuse; requires stronger evidence.",
    "violated_rules": ["abuse_03"],
    "confidence": 0.72,
    "human_decision": "OVERRIDE FLAG Keep for manual senior review",
    "final_action": "FLAG",
    "moderator_note": "Moderator override reason: Keep for manual senior review",
    "escalated": false
  },
  {
    "timestamp": "2026-06-28T09:30:00+00:00",
    "review_id": "REV-1003",
    "review_text": "Call me at 555-101-2211 for discounts and private deals.",
    "ai_recommendation": "FLAG",
    "ai_reasoning": "Contains potential personal/contact information and solicitation.",
    "violated_rules": ["privacy_07", "spam_02"],
    "confidence": 0.88,
    "human_decision": "ESCALATE Possible fraud ring indicator",
    "final_action": "ESCALATE",
    "moderator_note": "Possible fraud ring indicator",
    "escalated": true
  }
]
```

File: escalations.json


```
[
  {
    "timestamp": "2026-06-28T09:30:00+00:00",
    "review_id": "REV-1003",
```

```
    "review_text": "Call me at 555-101-2211 for discounts and private deals.",
    "ai_recommendation": "FLAG",
    "ai_reasoning": "Contains potential personal/contact information and solicitation.",
    "violated_rules": ["privacy_07", "spam_02"],
    "confidence": 0.88,
    "human_decision": "ESCALATE Possible fraud ring indicator",
    "final_action": "ESCALATE",
    "moderator_note": "Possible fraud ring indicator",
    "escalated": true
  }
]
```